

LangGraph Postgres Runtime

技术报告

开源版本差异分析与 postgres_py 实现详解

2026-06-15

目录

1. 项目背景与目标
2. 开源版本架构分析
3. 开源版本与生产态差异
4. postgres_py 架构设计
5. 模块详解: database.py
6. 模块详解: checkpoint.py
7. 模块详解: ops.py
8. 模块详解: store.py
9. 模块详解: run_queue.py
10. 模块详解: events.py & routes.py
11. 实现效果与测试结果
12. gRPC vs HTTP 版本差异
13. 待实现事项与总结

1. 项目背景与目标

1.1 LangGraph API 是什么

LangGraph API 是 LangChain 生态系统中的核心组件，提供了一个生产级的图执行平台。它允许开发者将基于 LangGraph 构建的 Agent 图部署为可扩展的 API 服务。

简单来说，LangGraph API 解决了以下问题：

- 持久化状态管理：图执行过程中的状态需要可靠存储，支持断点续传
- 并发执行：多个用户同时使用同一个 Agent，需要隔离的执行上下文
- 人机协作：图可以在关键节点暂停，等待人工审批后继续执行
- 异步处理：长时间运行的任务通过队列异步执行，不阻塞 API 响应

1.2 为什么需要 postgres_py 后端

LangGraph API 采用可插拔的后端架构，官方提供了两种后端：

- langgraph_runtime_inmem：内存存储，适合开发测试，无持久化
- langgraph_runtime_postgres (gRPC)：Go + Python 双进程架构，适合生产环境

然而，官方的 postgres 后端依赖于一个 Go 语言编写的 gRPC 服务，这给部署带来了额外的复杂性。我们的目标是实现一个纯 Python 的 Postgres 后端，既保持生产级特性，又简化部署。

1.3 项目目标

本项目 (langgraph_runtime_postgres_py) 的核心目标：

- 持久化存储：使用 PostgreSQL 存储图状态、线程、运行记录
- 分布式队列：使用 Redis Streams 实现任务队列，支持多 Worker
- 接口兼容：完全匹配 langgraph_api 期望的 ops 接口
- 纯 Python：无需额外进程，简化部署和运维

2. 开源版本架构分析

2.1 核心模块结构

langgraph_api 由以下核心模块组成：

模块	职责
api/	REST API 路由定义 (threads, runs, assistants, crons)
config/	配置解析 (DATABASE_URI, REDIS_URI, THREAD_TTL)
auth/	认证中间件 (noop, custom, langsmith)
grpc/	gRPC 客户端 ops (与 Go 服务通信)
worker/	图执行 Worker

2.2 Edition 检测机制

langgraph_api 通过环境变量 LANGGRAPH_RUNTIME_EDITION 决定使用哪个后端：

```
# feature_flags.py
_RUNTIME_EDITION = os.getenv('LANGGRAPH_RUNTIME_EDITION', 'inmem')
IS_POSTGRES_BACKEND = _RUNTIME_EDITION == 'postgres'
IS_POSTGRES_OR_GRPC_BACKEND = IS_POSTGRES_BACKEND
```

2.3 多态分发模式

API 路由根据 feature flag 选择不同的 ops 实现：

```
# api/threads.py
if IS_POSTGRES_OR_GRPC_BACKEND:
    from langgraph_api.grpc.ops import Threads # gRPC
else:
    from langgraph_runtime.ops import Threads # Python ops
```

这意味着：当 EDITION 不是 postgres/grpc 时，langgraph_runtime 会动态加载我们实现的后端包。

3. 开源版本与生产态差异

3.1 架构差异

特性	inmem	postgres (gRPC)	postgres_py (本项目)
持久化	□ 内存	□ PostgreSQL	□ PostgreSQL
分布式	□ 单进程	□ 多 Worker	□ 多 Worker
额外依赖	□ 无	□ Go 服务	□ 纯 Python
TTL 清理	□ 空壳	□ Go 后端	○ 部分
加密支持	□ 无	□ Go 后端	○ 部分

3.2 gRPC 架构详解

生产环境的 postgres 后端采用 Go + Python 双进程架构：

- Python 进程：运行 langgraph_api，处理 HTTP 请求，通过 gRPC 调用 Go 服务
- Go 进程：处理持久化操作、TTL 清理、加密等

这种架构的优势是性能和稳定性，但部署复杂度较高。

3.3 inmem 的局限性

- 状态存储在进程内存中，重启丢失
- 无法水平扩展，不支持多 Worker
- TTL sweep_ttl() 直接返回 (0, 0)，无实际清理
- 适合开发测试，不适合生产环境

4. postgres_py 架构设计

4.1 整体架构

本项目采用 PostgreSQL + Redis 双存储架构：

存储	用途	数据类型
PostgreSQL	持久化存储	threads, runs, assistants, crons, checkpoints, store_kv
Redis	分布式队列 + 事件总线	任务队列, 心跳, 事件流

4.2 生命周期管理

lifespan.py 负责启动和关闭所有组件：

```
async def lifespan(app):
    await start_pool() # PostgreSQL ████
    await start_checkpoint() # AsyncPostgresSaver
    await start_redis() # Redis ██
    await start_stream() # ██████
    await collect_graphs() # ██████
    # ██ Worker ███ Cron ███
    yield
    # █████...
```

4.3 数据库迁移

使用简单的版本化迁移机制：

- 001_initial_ops.sql: 核心表 (assistants, threads, runs, crons, worker_registry)
- 002_store.sql: KV 存储表 (store_kv)
- 003_schema_compat.sql: 兼容性字段 (expires_at, version, attempt)

5. 模块详解: database.py

5.1 连接池管理

使用 psycopg3 的 AsyncConnectionPool 管理数据库连接：

```
_pool = AsyncConnectionPool(
    conninfo=DATABASE_URI,
    max_size=POSTGRES_POOL_MAX_SIZE,
    kwargs={'row_factory': dict_row}
)
```

5.2 迁移机制

自动检测并执行未应用的迁移文件：

```
async def _run_migrations():
    current = await conn.execute(
        'SELECT COALESCE(MAX(v), 0) FROM runtime_migrations'
    )
    for fname in sorted(migration_files):
        if version > current:
            await cur.execute(sql)
            await conn.execute(
                'INSERT INTO runtime_migrations (v) VALUES (%s)',
```

```
(version,)
)
```

5.3 PgConnectionProto 适配器

封装 psycopg3 连接, 提供统一的接口供 ops 层使用:

- `execute(query, *args)`: 执行查询, 返回结果
- `pipeline()`: 批量操作优化

6. 模块详解: checkpoint.py

6.1 AsyncPostgresSaver 桥接

本模块是对 langgraph-checkpoint-postgres 的薄封装:

```
_checkpointer: AsyncPostgresSaver | None = None

def Checkpointer(*args, **kwargs) -> AsyncPostgresSaver:
    global _checkpointer
    if _checkpointer is None:
        raise RuntimeError('Call start_checkpointer() first')
    return _checkpointer
```

6.2 setup() 的 autocommit 处理

CREATE INDEX CONCURRENTLY 需要在 autocommit 模式下执行:

```
async def start_checkpointer():
    async with pool.connection() as conn:
        await conn.set_autocommit(True)
        saver = AsyncPostgresSaver(conn=conn)
        await saver.setup() # ■■■ checkpoint ■■■■
    _checkpointer = AsyncPostgresSaver(conn=pool)
```

7. 模块详解: ops.py

7.1 Authenticated 基类

所有 ops 类继承 Authenticated, 统一处理认证上下文:

```
class Authenticated:
    resource: Literal['threads', 'crons', 'assistants']

    @classmethod
    async def handle_event(cls, ctx, action, value):
        # ■■ langgraph_api.auth.custom.handle_event
        # ■■ auth filter ■■■■■■
```

7.2 返回类型规范

所有 CRUD 方法返回 AsyncIterator, 支持 fetchone() 和分页消费:

方法	返回类型	消费方式
put/patch/get/delete	AsyncIterator[dict]	fetchone()
search	tuple[AsyncIterator, int None]	get_pagination_headers()
count	int	直接返回

7.3 嵌套类

Threads.State: 线程状态操作 (get, post, bulk, list)

Threads.Stream: 线程事件流 (subscribe, join, publish)

Runs.Stream: 运行事件流 (subscribe, join, publish)

7.4 Worker 内部方法

方法	用途
Threads.set_status(conn, tid, checkpoint, exc)	更新线程状态
Threads.set_joint_status(...)	原子更新线程+运行状态
Runs.set_status(conn, rid, status)	更新运行状态
Runs.cancel(conn, run_ids, ...)	取消运行
Runs.enter(rid, tid, loop, resumable)	@asynccontextmanager 进入执行
Runs.next(wait, limit)	从队列获取待执行运行

8. 模块详解: store.py

8.1 PgStore 实现

实现 AsyncBatchedBaseStore 接口, 支持批量操作:

```
class PgStore(AsyncBatchedBaseStore):
    async def abatch(self, ops: Iterable[Op]) -> list[Any]:
```

```

for op in ops:
    if isinstance(op, PutOp):
        # INSERT ... ON CONFLICT
    elif isinstance(op, GetOp):
        # SELECT FROM store_kv
    elif isinstance(op, SearchOp):
        # SELECT + JSONB @> ■■

```

8.2 TTL 支持

支持过期时间，后台任务定期清理：

```

async def start_ttl_sweeper(self):
    while True:
        await conn.execute(
            'DELETE FROM store_kv WHERE expires_at < NOW()'
        )
        await asyncio.sleep(60)

```

9. 模块详解: run_queue.py

9.1 Redis Streams 消费者组

使用 Redis Streams 实现分布式任务队列：

```

RUNS_STREAM = 'lg:runs'
CONSUMER_GROUP = 'workers'

async def start_redis():
    await redis.xgroup_create(
        RUNS_STREAM, CONSUMER_GROUP, id='0', mkstream=True
    )

```

9.2 Worker 心跳

Worker 定期更新心跳，支持故障检测：

```

async def heartbeat_loop():
    await conn.execute(
        'INSERT INTO worker_registry (worker_id, ...) '
        'ON CONFLICT (worker_id) DO UPDATE '
        'SET last_heartbeat = NOW()'
    )

```

9.3 故障恢复

检测僵死 Worker 并重新分配任务：

```

await redis.xautoclaim(
    RUNS_STREAM, CONSUMER_GROUP, consumer_name,
    min_idle_time=STALE_WORKER_TIMEOUT_SECS * 1000
)

```

10. 模块详解: events.py & routes.py

10.1 Redis Pub/Sub 事件总线

发布订阅模式实现进程间事件通知：

```
class EventType(str, Enum):
    RUN_STARTED = 'run.started'
    RUN_COMPLETED = 'run.completed'
    RUN_FAILED = 'run.failed'
    THREAD_UPDATED = 'thread.updated'
    ...
```

10.2 内部管理路由

routes.py 提供调试和管理端点：

- /internal/truncate：清空所有数据（仅测试用）
- /internal/debug/thread/{thread_id}：查看线程详情

11. 实现效果与测试结果

11.1 Demo 测试

测试	内容	结果
Demo 1	Counter Graph (increment -> double 循环)	☐ 通过
Demo 2	Human-in-the-Loop (中断后恢复)	☐ 通过
Demo 3	完整 API 流程 (Assistant->Thread->Run->Events)	☐ 通过

11.2 PostgreSQL 数据验证

表	记录数	说明
assistants	2	注册的 counter + agent 图
checkpoints	59	完整的 checkpoint 链
checkpoint_writes	122	状态写入记录

Checkpoint 链验证：parent_checkpoint_id 关系正确建立，形成完整状态历史。

12. gRPC vs HTTP 版本差异

12.1 架构对比

维度	gRPC (生产)	HTTP (本项目)
进程模型	Go + Python 双进程	纯 Python 单进程
通信方式	gRPC + Protobuf	直接 Python 调用
部署复杂度	高 (需要 Go 服务)	低 (仅 Python)
性能	更高 (Go 处理持久化)	足够 (Python 直连 PG)
TTL 清理	Go 后端自动	需自己实现
加密	Go 后端处理	Python 层处理

12.2 ops 接口差异

gRPC ops 通过 protobuf 通信, Python ops 直接操作数据库:

```
# gRPC ops
async def Threads.get(conn, thread_id, ctx):
    client = await get_shared_client()
    response = await client.threads.Get(request)
    yield proto_to_thread(response)

# postgres_py ops
async def Threads.get(conn, thread_id, ctx):
    row = await conn.execute(
        'SELECT * FROM threads WHERE thread_id = %s',
        (thread_id,)
    )
    yield dict(row)
```

12.3 功能完整性

功能	gRPC	postgres_py	备注
CRUD 操作	☐	☐	完全对齐
状态管理	☐	☐	使用 AsyncPostgresSaver
任务队列	☐	☐	Redis Streams
TTL 清理	☐	○	需实现 sweep_ttl()
keep_latest 策略	☐	☐	待实现
加密支持	☐	○	部分实现

13. 待实现事项与总结

13.1 待实现事项

优先级	事项	说明
P0	实现 Threads.sweep_ttl()	扫描 expires_at < NOW() 的线程并删除
P0	启动 TTL 清理定时任务	在 lifespan 中每 sweep_interval_minutes 调用
P1	实现 keep_latest 策略	只清理旧 checkpoint, 保留最新状态
P1	优化 count() auth filter	当前加载全部行计数, 应推入 SQL
P2	Redis 显式清理 (可选)	在 Threads.delete() 中清理 run stream

13.2 项目总结

本项目成功实现了一个纯 Python 的 LangGraph Postgres 后端，具有以下特点：

- 持久化存储：使用 PostgreSQL 存储 threads, runs, assistants, checkpoints
- 分布式队列：使用 Redis Streams 实现多 Worker 任务调度
- 接口兼容：完全匹配 langgraph_api 期望的 ops 接口
- 部署简化：无需额外的 Go 服务，降低运维复杂度

通过本项目，开发者可以在不引入 gRPC 架构的情况下，获得生产级的持久化能力。